

Practical 1 - MongoDB Basics:1

A. Write a MongoDB query to create and drop database.

```
> show dbs
Employee      0.000GB
Romal_239720  0.000GB
admin         0.000GB
config        0.000GB
local         0.000GB
> use Romal
switched to db Romal
> show dbs
Employee      0.000GB
Romal_239720  0.000GB
admin         0.000GB
config        0.000GB
local         0.000GB
> db.createCollection("Practical1")
{ "ok" : 1 }
> show collections
Practical1
> show dbs
Employee      0.000GB
Romal         0.000GB
Romal_239720  0.000GB
admin         0.000GB
config        0.000GB
local         0.000GB
> db.dropDatabase()
{ "ok" : 1 }
```

B. Write a MongoDB query to create, display and drop collection.

```
> use Romal
switched to db Romal
> show collections
> db.createCollection("Practical1")
{ "ok" : 1 }
> show collections
Practical1
> db.Practical1.drop()
true
```

C. Write a MongoDB query to insert, query, update and delete a document.

```
> use Romal
switched to db Romal
> db.Doc.insert({Name:"Romal",Age:10})
WriteResult({ "nInserted" : 1 })
> db.Doc.find()
{ "_id" : ObjectId("652b72eb7c19cf3ece5423bc"), "Name" : "Romal", "Age" : 10 }
> db.Doc.update({Name:"Romal"},{$set:{Age:20}})
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
> db.Doc.find()
{ "_id" : ObjectId("652b72eb7c19cf3ece5423bc"), "Name" : "Romal", "Age" : 20 }
> db.Doc.remove()
uncaught exception: Error: remove needs a query :
DBCollection.prototype._parseRemove@src/mongo/shell/collection.js:364:15
DBCollection.prototype.remove@src/mongo/shell/collection.js:394:18
@(shell):1:1
> db.Doc.remove({"Name":"Romal"})
WriteResult({ "nRemoved" : 1 })
> db.Doc.find()
> _
```

D. Employee Queries

1) Insert and display the documents in the Employee collection.

```
> use Employee
switched to db Employee
> db.col.insertMany([{Name:"Smitu",Salary:100000,Dept:"ID",City:"Mumbai"},{Name:"Romal",Salary:15000,Dept:"IT",City:"Thane"},{Name:"Sujaal",Salary:9000,Dept:"Tester",City:"Bandra"},{Name:"Manav",Salary:30000,Dept:"Finance",City:"Kurla"},{Name:"Swathi",Salary:4000,Dept:"Developer",City:"Pune"}])
{
  "acknowledged" : true,
  "insertedIds" : [
    ObjectId("652b7f079088ab5f16b0dac0"),
    ObjectId("652b7f079088ab5f16b0dac1"),
    ObjectId("652b7f079088ab5f16b0dac2"),
    ObjectId("652b7f079088ab5f16b0dac3"),
    ObjectId("652b7f079088ab5f16b0dac4")
  ]
}
> db.col.find()
{ "_id" : ObjectId("652b7f079088ab5f16b0dac0"), "Name" : "Smitu", "Salary" : 100000, "Dept" : "ID", "City" : "Mumbai" }
{ "_id" : ObjectId("652b7f079088ab5f16b0dac1"), "Name" : "Romal", "Salary" : 15000, "Dept" : "IT", "City" : "Thane" }
{ "_id" : ObjectId("652b7f079088ab5f16b0dac2"), "Name" : "Sujaal", "Salary" : 9000, "Dept" : "Tester", "City" : "Bandra" }
{ "_id" : ObjectId("652b7f079088ab5f16b0dac3"), "Name" : "Manav", "Salary" : 30000, "Dept" : "Finance", "City" : "Kurla" }
```

2) Display details of employees who live in Mumbai.

```
> db.col.find({City:"Mumbai"})
{ "_id" : ObjectId("652b7f079088ab5f16b0dac0"), "Name" : "Smitu", "Salary" : 100000, "Dept" : "ID", "City" : "Mumbai" }
```

3) Display details of employee staying in Kurla and salary greater than 19000

```
> db.col.find({City:"Kurla",Salary:{$gt:19000}})
{ "_id" : ObjectId("652b7f079088ab5f16b0dac3"), "Name" : "Manav", "Salary" : 30000, "Dept" : "Finance", "City" : "Kurla" }
```

4) Display details of the employee whose salary is greater than equal 10000 but less than 20000.

```
> db.col.find({Salary:{$gte:10000, $lt:20000}})
{ "_id" : ObjectId("652b7f079088ab5f16b0dac1"), "Name" : "Romal", "Salary" : 15000, "Dept" : "IT", "City" : "Thane" }
```

5) Display details of employees staying in either thane or mumbai and salary greater than 10000.

```
> db.col.find({$or:[{City:"Mumbai"},{City:"Thane"}],Salary:{$gt:10000}})
{ "_id" : ObjectId("652b7f079088ab5f16b0dac0"), "Name" : "Smitu", "Salary" : 100000, "Dept" : "ID", "City" : "Mumbai" }
{ "_id" : ObjectId("652b7f079088ab5f16b0dac1"), "Name" : "Romal", "Salary" : 15000, "Dept" : "IT", "City" : "Thane" }
```

6) Display details of employee whose salary is not equal to 10000 and working in either developer or tester department.

```
> db.col.find({$or:[{Dept:"Developer"},{Dept:"Tester"}],Salary:{$ne:10000}})
{ "_id" : ObjectId("652b7f079088ab5f16b0dac2"), "Name" : "Sujal", "Salary" : 9000, "Dept" : "Tester", "City" : "Bandra" }
{ "_id" : ObjectId("652b7f079088ab5f16b0dac4"), "Name" : "Swathi", "Salary" : 4000, "Dept" : "Developer", "City" : "Pune" }
```

2. Practical 2 - MongoDB Basics:2

3. Insert the documents

```
> use Data
switched to db Data
> db.col.insertMany([
... {Name: "Roma", Age:20, Country:"India", Gender:"Female"},
... {Name: "Raj", Age:12, Country:"Russia", Gender:"Male"},
... {Name: "Suja", Age:25, Country:"USA", Gender:"Male"},
... {Name: "Sara", Age:65, Country:"India", Gender:"Female"},
... {Name: "Koyal", Age:30, Country:"USA", Gender:"Female"}
... ])
{
  "acknowledged" : true,
  "insertedIds" : [
    ObjectId("652b8b379088ab5f16b0dac5"),
    ObjectId("652b8b379088ab5f16b0dac6"),
    ObjectId("652b8b379088ab5f16b0dac7"),
    ObjectId("652b8b379088ab5f16b0dac8"),
    ObjectId("652b8b379088ab5f16b0dac9")
  ]
}
> db.col.find()
{ "_id" : ObjectId("652b8b379088ab5f16b0dac5"), "Name" : "Roma", "Age" : 20, "Country" : "India", "Gender" : "Female" }
{ "_id" : ObjectId("652b8b379088ab5f16b0dac6"), "Name" : "Raj", "Age" : 12, "Country" : "Russia", "Gender" : "Male" }
{ "_id" : ObjectId("652b8b379088ab5f16b0dac7"), "Name" : "Suja", "Age" : 25, "Country" : "USA", "Gender" : "Male" }
{ "_id" : ObjectId("652b8b379088ab5f16b0dac8"), "Name" : "Sara", "Age" : 65, "Country" : "India", "Gender" : "Female" }
{ "_id" : ObjectId("652b8b379088ab5f16b0dac9"), "Name" : "Koyal", "Age" : 30, "Country" : "USA", "Gender" : "Female" }
```

4. Update the country to UK for all female users.

```
> db.col.updateMany({Gender:"Female"},{$set:{Country:"UK"}})
{ "acknowledged" : true, "matchedCount" : 3, "modifiedCount" : 3 }
> db.col.find()
{ "_id" : ObjectId("652b8b379088ab5f16b0dac5"), "Name" : "Roma", "Age" : 20, "Country" : "UK", "Gender" : "Female" }
{ "_id" : ObjectId("652b8b379088ab5f16b0dac6"), "Name" : "Raj", "Age" : 12, "Country" : "Russia", "Gender" : "Male" }
{ "_id" : ObjectId("652b8b379088ab5f16b0dac7"), "Name" : "Suja", "Age" : 25, "Country" : "USA", "Gender" : "Male" }
{ "_id" : ObjectId("652b8b379088ab5f16b0dac8"), "Name" : "Sara", "Age" : 65, "Country" : "UK", "Gender" : "Female" }
{ "_id" : ObjectId("652b8b379088ab5f16b0dac9"), "Name" : "Koyal", "Age" : 30, "Country" : "UK", "Gender" : "Female" }
```

5. Add the new field company to all the documents.

```
> db.col.updateMany({},{$set:{Company:"Wipro"}})
{ "acknowledged" : true, "matchedCount" : 5, "modifiedCount" : 5 }
> db.col.find()
{ "_id" : ObjectId("652b8b379088ab5f16b0dac5"), "Name" : "Roma", "Age" : 20, "Country" : "UK", "Gender" : "Female", "Company" : "Wipro" }
{ "_id" : ObjectId("652b8b379088ab5f16b0dac6"), "Name" : "Raj", "Age" : 12, "Country" : "Russia", "Gender" : "Male", "Company" : "Wipro" }
{ "_id" : ObjectId("652b8b379088ab5f16b0dac7"), "Name" : "Suja", "Age" : 25, "Country" : "USA", "Gender" : "Male", "Company" : "Wipro" }
{ "_id" : ObjectId("652b8b379088ab5f16b0dac8"), "Name" : "Sara", "Age" : 65, "Country" : "UK", "Gender" : "Female", "Company" : "Wipro" }
{ "_id" : ObjectId("652b8b379088ab5f16b0dac9"), "Name" : "Koyal", "Age" : 30, "Country" : "UK", "Gender" : "Female", "Company" : "Wipro" }
```

6. Delete all the documents where Gender = 'M'.

```
> db.col.remove({"Gender":"Male"})
WriteResult({ "nRemoved" : 2 })
> db.col.find()
{ "_id" : ObjectId("652b8b379088ab5f16b0dac5"), "Name" : "Roma1", "Age" : 20, "Country" : "UK",
"Gender" : "Female", "Company" : "Wipro" }
{ "_id" : ObjectId("652b8b379088ab5f16b0dac8"), "Name" : "Sara", "Age" : 65, "Country" : "UK", "
Gender" : "Female", "Company" : "Wipro" }
{ "_id" : ObjectId("652b8b379088ab5f16b0dac9"), "Name" : "Koyal", "Age" : 30, "Country" : "UK",
"Gender" : "Female", "Company" : "Wipro" }
>
```

1. Find out a count of female users who stay in either India or USA.

```
> db.col.find({$or:[{Country:"India"},{Country:"USA"}],Gender:"Female"}).count()
0
```

2. Display the first name and age of all female employees.

```
> db.col.find({Gender:"Female"},{Name:1,Age:1})
{ "_id" : ObjectId("652b8b379088ab5f16b0dac5"), "Name" : "Roma1", "Age" : 20 }
{ "_id" : ObjectId("652b8b379088ab5f16b0dac8"), "Name" : "Sara", "Age" : 65 }
{ "_id" : ObjectId("652b8b379088ab5f16b0dac9"), "Name" : "Koyal", "Age" : 30 }
>
```

Practical 3 – Aggregate Functions

Insert collection in the database and display all the records.

```
> use Aggregate
switched to db Aggregate
> db.col.insertMany([ {Name: "Romal",Age:17,City:"Mumbai",Salary:12000}, {Name: "Rohan",Age:22,City:"Thane",Salary:60000},
{Name: "Sujaal",Age:19,City:"Mumbai",Salary:3000}, {Name: "Manav",Age:47,City:"Pune",Salary:55000}, {Name: "Ria",Age:34,Ci
ty:"Mumbai",Salary:83000} ])
{
  "acknowledged" : true,
  "insertedIds" : [
    ObjectId("652bac539088ab5f16b0dacf"),
    ObjectId("652bac539088ab5f16b0dad0"),
    ObjectId("652bac539088ab5f16b0dad1"),
    ObjectId("652bac539088ab5f16b0dad2"),
    ObjectId("652bac539088ab5f16b0dad3")
  ]
}
> db.col.find()
{ "_id" : ObjectId("652bac539088ab5f16b0dacf"), "Name" : "Romal", "Age" : 17, "City" : "Mumbai", "Salary" : 12000 }
{ "_id" : ObjectId("652bac539088ab5f16b0dad0"), "Name" : "Rohan", "Age" : 22, "City" : "Thane", "Salary" : 60000 }
{ "_id" : ObjectId("652bac539088ab5f16b0dad1"), "Name" : "Sujaal", "Age" : 19, "City" : "Mumbai", "Salary" : 3000 }
{ "_id" : ObjectId("652bac539088ab5f16b0dad2"), "Name" : "Manav", "Age" : 47, "City" : "Pune", "Salary" : 55000 }
{ "_id" : ObjectId("652bac539088ab5f16b0dad3"), "Name" : "Ria", "Age" : 34, "City" : "Mumbai", "Salary" : 83000 }
```

1. Group by function to get count.

```
> db.col.aggregate([{$group:{_id:"$City",cityCount:{$sum:1}}}]])
{ "_id" : "Thane", "cityCount" : 1 }
{ "_id" : "Mumbai", "cityCount" : 3 }
{ "_id" : "Pune", "cityCount" : 1 }
```

2. Sum function.

```
> db.col.aggregate([{$group:{_id:"$City",cityCount:{$sum:"$Salary"}}}]])
{ "_id" : "Thane", "cityCount" : 60000 }
{ "_id" : "Mumbai", "cityCount" : 98000 }
{ "_id" : "Pune", "cityCount" : 55000 }
```

3. Avg function.

```
> db.col.aggregate([{$group:{_id:"$City",cityCount:{$avg:"$Salary"}}}]])
{ "_id" : "Thane", "cityCount" : 60000 }
{ "_id" : "Mumbai", "cityCount" : 32666.666666666668 }
{ "_id" : "Pune", "cityCount" : 55000 }
```

1. Min function
2. Max function.
3. First function
4. Last function

```
> db.col.aggregate([{$group: {_id: "$City", cityCount: {$min: "$Salary"}}}])
{ "_id" : "Thane", "cityCount" : 60000 }
{ "_id" : "Mumbai", "cityCount" : 3000 }
{ "_id" : "Pune", "cityCount" : 55000 }
> db.col.aggregate([{$group: {_id: "$City", cityCount: {$max: "$Salary"}}}])
{ "_id" : "Thane", "cityCount" : 60000 }
{ "_id" : "Mumbai", "cityCount" : 83000 }
{ "_id" : "Pune", "cityCount" : 55000 }
> db.col.aggregate([{$group: {_id: "$City", cityCount: {$first: "$Salary"}}}])
{ "_id" : "Thane", "cityCount" : 60000 }
{ "_id" : "Mumbai", "cityCount" : 12000 }
{ "_id" : "Pune", "cityCount" : 55000 }
> db.col.aggregate([{$group: {_id: "$City", cityCount: {$last: "$Salary"}}}])
{ "_id" : "Thane", "cityCount" : 60000 }
{ "_id" : "Mumbai", "cityCount" : 83000 }
{ "_id" : "Pune", "cityCount" : 55000 }
```

5. Push function
6. addToSet function

```
> db.col.aggregate([{$group: {_id: "$City", cityCount: {$addToSet: "$Salary"}}}])
{ "_id" : "Thane", "cityCount" : [ 60000 ] }
{ "_id" : "Mumbai", "cityCount" : [ 3000, 12000, 83000 ] }
{ "_id" : "Pune", "cityCount" : [ 55000 ] }
> db.col.aggregate([{$group: {_id: "$City", cityCount: {$push: "$Salary"}}}])
{ "_id" : "Mumbai", "cityCount" : [ 12000, 3000, 83000 ] }
{ "_id" : "Pune", "cityCount" : [ 55000 ] }
{ "_id" : "Thane", "cityCount" : [ 60000 ] }
```

4. Java and MongoDB

1. Insert the document in MongoDB using Java.

```
import com.mongodb.client.MongoCollection;
import com.mongodb.client.MongoDatabase;
import com.mongodb.MongoClient;
import org.bson.Document;
public class insert
{
    public static void main(String[] args)
    {
        MongoClient mongo = new MongoClient("localhost",27017);
        System.out.println("Connected to the database successfully.");
        MongoDatabase database = mongo.getDatabase("TYITDB239720");
        MongoCollection<Document> collection = database.getCollection("myCol");
        System.out.println("Collection myCol selected successfully");
        Document document = new Document();
        document.append("id",1);
        document.append("Name","Romal");
        document.append("RollNo",239720);
        document.append("Age",20);
        document.append("College","MCC");
        collection.insertOne(document);
        System.out.println("Document inserted successfully");
    }
}
```

Output:

```
C:\Users\Admin\Desktop\MongoDB Practical>java insert
Jul 26, 2023 3:50:18 PM com.mongodb.diagnostics.logging.JULLogger log
INFO: Cluster created with settings {hosts=[localhost:27017], mode=SINGLE, requiredClusterType=UNKNOWN,
serverSelectionTimeout='30000 ms', maxWaitQueueSize=500}
Connected to the database successfully.
Collection myCol selected successfully
Jul 26, 2023 3:50:18 PM com.mongodb.diagnostics.logging.JULLogger log
INFO: No server chosen by PrimaryServerSelector from cluster description ClusterDescription{type=UNKNOWN,
connectionMode=SINGLE, all=[ServerDescription{address=localhost:27017, type=UNKNOWN, state=CONNECTING}]].
Waiting for 30000 ms before timing out
Jul 26, 2023 3:50:18 PM com.mongodb.diagnostics.logging.JULLogger log
INFO: Opened connection [connectionId{localValue:1, serverValue:11}] to localhost:27017
Jul 26, 2023 3:50:18 PM com.mongodb.diagnostics.logging.JULLogger log
INFO: Monitor thread successfully connected to server with description ServerDescription{address=localhost:27017,
type=STANDALONE, state=CONNECTED, ok=true, version=ServerVersion{versionList=[5, 0, 18]}, minWireVersion=0,
maxWireVersion=13, electionId=null, maxDocumentSize=16777216, roundTripTimeNanos=559600}
Jul 26, 2023 3:50:18 PM com.mongodb.diagnostics.logging.JULLogger log
INFO: Opened connection [connectionId{localValue:2, serverValue:12}] to localhost:27017
Document inserted successfully
```

```
> show dbs
123456          0.000GB
34_mydb         0.000GB
TYITDB239720    0.000GB
admin           0.000GB
config          0.000GB
local           0.000GB
romaldb         0.000GB
> use TYITDB239720
switched to db TYITDB239720
> show collections
myCol
> db.myCol.find().pretty()
{
  "_id" : ObjectId("64c0b6be088d4125188111d1"),
  "id" : 1,
  "Name" : "Roma1",
  "RollNo" : 239720,
  "Age" : 20,
  "College" : "MCC"
}
```

1. Update the document in MongoDB using Java.

```
import com.mongodb.DB;
import com.mongodb.DBCollection;
import com.mongodb.BasicDBObject;
import com.mongodb.MongoClient;
import com.mongodb.WriteResult;
import java.net.UnknownHostException;
public class update
{
    public static void main(String[] args)
    {
        MongoClient mongo = new MongoClient("localhost",27017);
        System.out.println("Connected to the database successfully.");
        DB db = mongo.getDB("TYITDB239720");
        DBCollection col = db.getCollection("myCol");
        BasicDBObject query = new BasicDBObject("id",1);
        BasicDBObject update = new BasicDBObject();
        update.put("$set",new BasicDBObject("Age",27));
        WriteResult result = col.update(query,update);
        mongo.close();
    }
}
```

Output:

```
C:\Users\Admin\Desktop\MongoDB Practical>javac update.java
Note: update.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

C:\Users\Admin\Desktop\MongoDB Practical>java update
Jul 26, 2023 3:38:52 PM com.mongodb.diagnostics.logging.JULLogger log
INFO: Cluster created with settings {hosts=[localhost:27017], mode=SINGLE, requiredClusterType=UNKNOWN, serverSelectionTimeout='30000 ms', maxWaitQueueSize=500}
Connected to the database successfully.
Jul 26, 2023 3:38:52 PM com.mongodb.diagnostics.logging.JULLogger log
INFO: No server chosen by PrimaryServerSelector from cluster description ClusterDescription{type=UNKNOWN, connectionMode=SINGLE, all=[ServerDescription{address=localhost:27017, type=UNKNOWN, state=CONNECTING}]]. Waiting for 30000 ms before timing out
Jul 26, 2023 3:38:52 PM com.mongodb.diagnostics.logging.JULLogger log
INFO: Opened connection [connectionId{localValue:1, serverValue:5}] to localhost:27017
Jul 26, 2023 3:38:52 PM com.mongodb.diagnostics.logging.JULLogger log
INFO: Monitor thread successfully connected to server with description ServerDescription{address=localhost:27017, type=STANDALONE, state=CONNECTED, ok=true, version=ServerVersion{versionList=[5, 0, 18]}, minWireVersion=0, maxWireVersion=13, electionId=null, maxDocumentSize=16777216, roundTripTimeNanos=1035400}
Jul 26, 2023 3:38:52 PM com.mongodb.diagnostics.logging.JULLogger log
INFO: Opened connection [connectionId{localValue:2, serverValue:6}] to localhost:27017
Jul 26, 2023 3:38:53 PM com.mongodb.diagnostics.logging.JULLogger log
INFO: Closed connection [connectionId{localValue:2, serverValue:6}] to localhost:27017 because the pool has been closed.
```

```
> db.myCol.find().pretty()
{
  "_id" : ObjectId("64c0b6be088d4125188111d1"),
  "id" : 1,
  "Name" : "Roma1",
  "RollNo" : 239720,
  "Age" : 27,
  "College" : "MCC"
}
```

1. Retrieve the document in MongoDB using Java

```
import com.mongodb.client.MongoCollection;
import com.mongodb.client.MongoDatabase;
import com.mongodb.MongoClient;
import com.mongodb.MongoCredential;
import org.bson.Document;
import java.util.Iterator;
import com.mongodb.client.FindIterable;

public class retrieve
{
    public static void main(String[] args)
    {
        MongoClient mongo = new MongoClient("localhost",27017);
        System.out.println("Connected to the database successfully.");
        MongoDatabase database = mongo.getDatabase("TYITDB239720");
        MongoCollection<Document> collection = database.getCollection("myCol");
        System.out.println("Collection myCol selected successfully");
        FindIterable<Document> iterDoc = collection.find();
        int i = 1;
        Iterator it = iterDoc.iterator();
        while(it.hasNext())
        {
            System.out.println(it.next());
            i++;
        }
    }
}
```

Output:

```
C:\Users\Admin\Desktop\MongoDB Practical>javac retrieve.java

C:\Users\Admin\Desktop\MongoDB Practical>java retrieve
Jul 26, 2023 3:44:34 PM com.mongodb.diagnostics.logging.JULLogger log
INFO: Cluster created with settings {hosts=[localhost:27017], mode=SINGLE, requiredClusterType=UNKNOWN,
serverSelectionTimeout='30000 ms', maxWaitQueueSize=500}
Connected to the database successfully.
Collection myCol selected successfully
Jul 26, 2023 3:44:34 PM com.mongodb.diagnostics.logging.JULLogger log
INFO: No server chosen by ReadPreferenceServerSelector{readPreference=primary} from cluster description
ClusterDescription{type=UNKNOWN, connectionMode=SINGLE, all=[ServerDescription{address=localhost:27017
, type=UNKNOWN, state=CONNECTING}]]. Waiting for 30000 ms before timing out
Jul 26, 2023 3:44:34 PM com.mongodb.diagnostics.logging.JULLogger log
INFO: Opened connection [connectionId{localValue:1, serverValue:9}] to localhost:27017
Jul 26, 2023 3:44:34 PM com.mongodb.diagnostics.logging.JULLogger log
INFO: Monitor thread successfully connected to server with description ServerDescription{address=localh
ost:27017, type=STANDALONE, state=CONNECTED, ok=true, version=ServerVersion{versionList=[5, 0, 18]}, mi
nWireVersion=0, maxWireVersion=13, electionId=null, maxDocumentSize=16777216, roundTripTimeNanos=105240
0}
Jul 26, 2023 3:44:34 PM com.mongodb.diagnostics.logging.JULLogger log
INFO: Opened connection [connectionId{localValue:2, serverValue:10}] to localhost:27017
Document({_id=64c0b6be088d4125188111d1, id=1, Name=Romal, RollNo=239720, Age=27, College=MCC})

INFO: Opened connection [connectionId{localValue:2, serverValue:10}] to localhost:27017
Document({_id=64c0b6be088d4125188111d1, id=1, Name=Romal, RollNo=239720, Age=27, College=MCC})
```

1. Delete the document in MongoDB using Java.

```
import com.mongodb.client.MongoCollection;
import com.mongodb.client.MongoDatabase;
import com.mongodb.MongoClient;
import com.mongodb.MongoCredential;
import org.bson.Document;
import com.mongodb.client.model.Filters;
public class delete
{
    public static void main(String[] args)
    {
        MongoClient mongo = new MongoClient("localhost",27017);
        System.out.println("Connected to the database successfully.");
        MongoDatabase database = mongo.getDatabase("TYITDB239720");
        MongoCollection<Document> collection = database.getCollection("myCol");
        System.out.println("Collection myCol selected successfully");
        collection.deleteOne(Filters.eq("id",1));
        System.out.println("Document deleted successfully");
    }
}
```

Output:

```
C:\Users\Admin\Desktop\MongoDB Practical>javac delete.java
```

```
C:\Users\Admin\Desktop\MongoDB Practical>java delete
```

```
Jul 26, 2023 4:02:55 PM com.mongodb.diagnostics.logging.JULLogger log
INFO: Cluster created with settings {hosts=[localhost:27017], mode=SINGLE, requiredClusterType=UNKNOWN,
serverSelectionTimeout='30000 ms', maxWaitQueueSize=500}
Connected to the database successfully.
Collection myCol selected successfully
Jul 26, 2023 4:02:55 PM com.mongodb.diagnostics.logging.JULLogger log
INFO: No server chosen by PrimaryServerSelector from cluster description ClusterDescription{type=UNKNOWN,
connectionMode=SINGLE, all=[ServerDescription{address=localhost:27017, type=UNKNOWN, state=CONNECTING}]].
Waiting for 30000 ms before timing out
Jul 26, 2023 4:02:55 PM com.mongodb.diagnostics.logging.JULLogger log
INFO: Opened connection [connectionId{localValue:1, serverValue:13}] to localhost:27017
Jul 26, 2023 4:02:55 PM com.mongodb.diagnostics.logging.JULLogger log
INFO: Monitor thread successfully connected to server with description ServerDescription{address=localhost:27017,
type=STANDALONE, state=CONNECTED, ok=true, version=ServerVersion{versionList=[5, 0, 18]}, minWireVersion=0,
maxWireVersion=13, electionId=null, maxDocumentSize=16777216, roundTripTimeNanos=781300}
Jul 26, 2023 4:02:55 PM com.mongodb.diagnostics.logging.JULLogger log
INFO: Opened connection [connectionId{localValue:2, serverValue:14}] to localhost:27017
Document deleted successfully
```

```
INFO: Opened connection [connectionId{localValue:2, serverValue:14}] to localhost:27017
Document deleted successfully
```

```
> db.myCol.find().pretty()
>
```

5.PHP

php_mongo.dll ---> copy this file and paste it in xampp--> php--> ext

Edit the configuration --> xampp --> php ---> php(Type:
Configuration setting)

```
extension=php_mysql.dll
;extension=php_oci8_12c.dll ; Use with
Instant Client
;extension=php_openssl.dll
;extension=php_pdo_firebird.dll
extension=php_pdo_mysql.dll
;extension=php_pdo_oci.dll
;extension=php_pdo_odbc.dll
;extension=php_pdo_pgsql.dll
extension=php_pdo_sqlite.dll
;extension=php_pgsql.dll
;extension=php_shmop.dll
extension=php_mongo.dll
```

```
; The MIBS data available in the PHP dis
installed.
```

```
; See http://www.php.net/manual/en/snmp.
;extension=php_snmp.dll
```

```
;extension=php_soap.dll
```

Save Files at the location:

C:\xampp\htdocs\

Execute:

Open xampp

Apache → Start (Make it active) and Admin(Make it active)

Browser will open

Remove dashboard and instead give the name of the file (eg: insert.php)

Output is printed.

insert.php

```
<?php
```

```
$m = new MongoClient();
```

```
echo "Connection to database successfully";
```

```
$db = $m -> MYDB239720;
```

```
echo "\nDatabase selected successfully";
```

```
$col = $db -> MyCol;
```

```
echo "\nCollection selected successfully";
```

```
$doc = array(
```

```
"name" => "Romal",
```

```
"age" => 20,
```

```
"dept" => "TYIT",
```

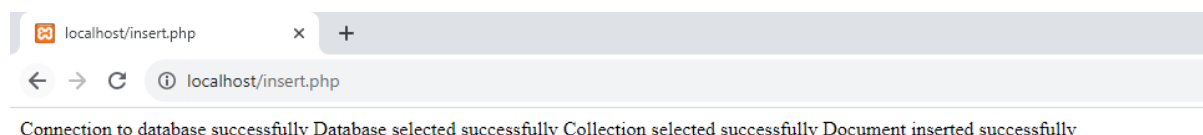
```
"rollno" => 239720
```

```
);
```

```
$col -> insert($doc);
```

```
echo "\nDocument inserted successfully";
```

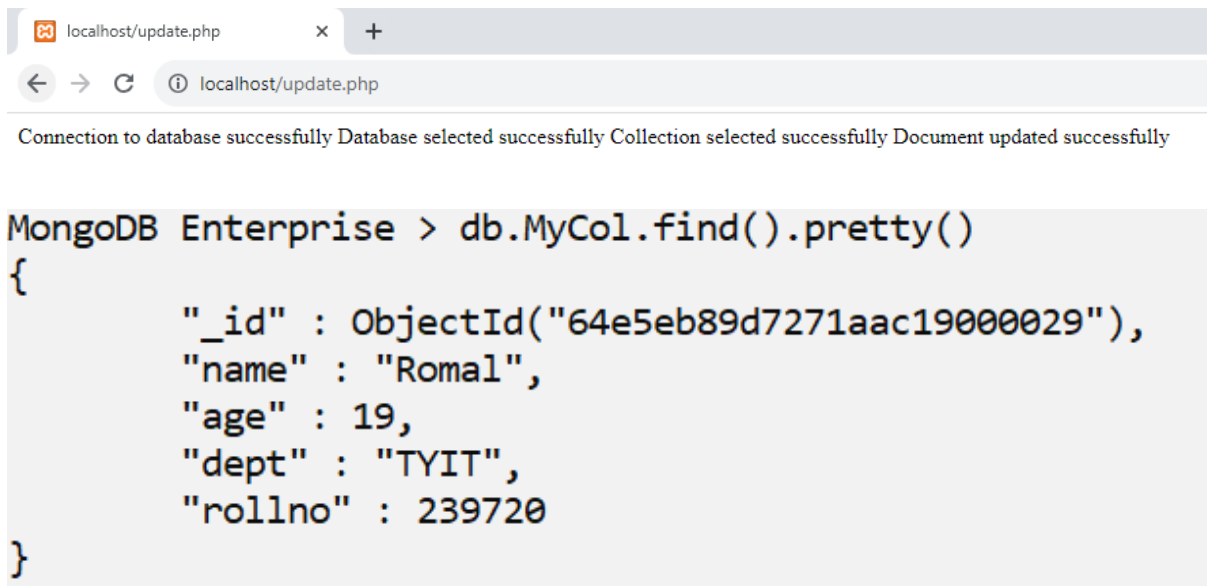
```
?>
```



```
MongoDB Enterprise > show dbs
MYDB239720  0.000GB
TYIT239720  0.001GB
admin       0.000GB
config      0.000GB
local       0.000GB
MongoDB Enterprise > use MYDB239720
switched to db MYDB239720
MongoDB Enterprise > show collections
MyCol
MongoDB Enterprise > db.MyCol.find().pretty()
{
  "_id" : ObjectId("64e5eb89d7271aac19000029"),
  "name" : "Romal",
  "age" : 20,
  "dept" : "TYIT",
  "rollno" : 239720
}
```

2. update.php

```
<?php
$m = new MongoClient();
echo "Connection to database successfully";
$db = $m -> MYDB239720;
echo "\nDatabase selected successfully";
$col = $db -> MyCol;
echo "\nCollection selected successfully";
$col -> update(array( "name" => "Romal"),array('$set'=>array("age" => 19)));
echo "\nDocument updated successfully";
?>
```



The screenshot shows a web browser window with the address bar at `localhost/update.php`. Below the browser, a status message reads: "Connection to database successfully Database selected successfully Collection selected successfully Document updated successfully". Below this, a MongoDB shell session shows the command `db.MyCol.find().pretty()` and its output, which is a JSON document: `{ "_id" : ObjectId("64e5eb89d7271aac19000029"), "name" : "Romal", "age" : 19, "dept" : "TYIT", "rollno" : 239720 }`.

```
MongoDB Enterprise > db.MyCol.find().pretty()
{
  "_id" : ObjectId("64e5eb89d7271aac19000029"),
  "name" : "Romal",
  "age" : 19,
  "dept" : "TYIT",
  "rollno" : 239720
}
```

3. retrieve.php

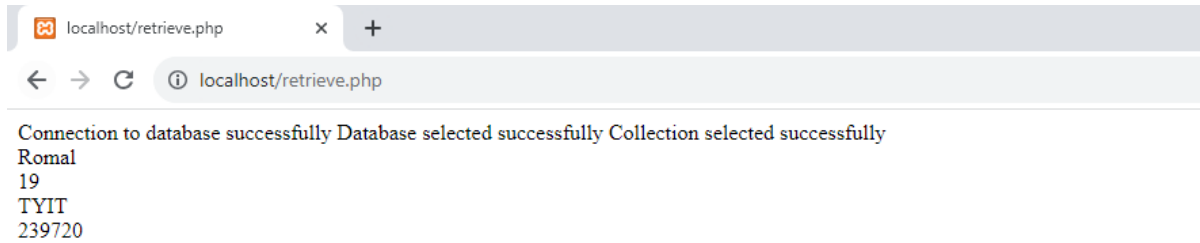
```
<?php
$m = new MongoClient();
echo "Connection to database successfully";

$db = $m -> MYDB239720;
echo "\nDatabase selected successfully";

$col = $db -> MyCol;
echo "\nCollection selected successfully";

$cursor = $col -> find();
foreach($cursor as $doc)
{
    echo"<br/>";
    echo $doc["name"]."<br/>";
    echo $doc["age"]."<br/>";
    echo $doc["dept"]."<br/>";
    echo $doc["rollno"]."<br/>";
}
```

?>

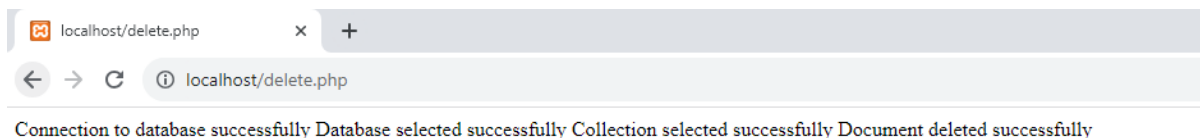


4. delete.php

<?php

```
$m = new MongoClient();  
echo "Connection to database successfully";  
$db = $m -> MYDB239720;  
echo "\nDatabase selected successfully";  
$col = $db -> MyCol;  
echo "\nCollection selected successfully";  
$col -> remove(array( "name" => "Romal"));  
echo "\nDocument deleted successfully";
```

?>



```
MongoDB Enterprise > db.MyCol.find().pretty()  
MongoDB Enterprise > █
```

6.Python

1.insert

```
from pymongo import MongoClient
client = MongoClient('localhost',27017)
db = client.TYITDB239720
def insert():
    try:
        name1 = input("Enter the Name:")
        age1 = input("Enter the Age:")
        dept1 = input("Enter the Department:")
        pin1 = input("Enter the Pin No:")
        db.MyCol.insert_one({
            "name":name1,
            "age":age1,
            "dept":dept1,
            "pin":pin1
        })
        print("Inserted data successfully")
    except Exception:
        print(str(e))
```

insert()

Enter the Name:Romal

Enter the Age:19

Enter the Department:I.T.

Enter the Pin No:239720

Inserted data successfully

```
> show dbs
123456      0.000GB
34_mydb     0.000GB
TYITDB239720 0.000GB
admin       0.000GB
config      0.000GB
local       0.000GB
romaldb     0.000GB
> use TYITDB239720
switched to db TYITDB239720
> show collections
MyCol
> db.MyCol.find()
{ "_id" : ObjectId("64d08b585d7a653a599a17b9"), "name" : "Romal", "age" : "19", "dept" : "I.T.",
  "pin" : "239720" }
> db.MyCol.find()
{ "_id" : ObjectId("64d08b585d7a653a599a17b9"), "name" : "Romal", "age" : "19", "dept" : "I.T.",
  "pin" : "239720" }
```

2. update()

```
from pymongo import MongoClient
```

```
client = MongoClient('localhost',27017)
```

```
db = client.TYITDB239720
```

```
def update():
```

```
    try:
```

```
        name1 = input("Enter the Name:")
```

```
        age1 = input("Enter the Age to update:")
```

```
        db.MyCol.update_one({"name":name1},{"$set":{"age":age1}})
```

```
        print("\nRecords updated successfully\n")
```

```
    except Exception:
```

```
        print(str(e))
```

```
update()
```

Enter the Name: Romal

Enter the Age to update: 20

Records updated successfully

```
> db.MyCol.find()
{ "_id" : ObjectId("64d08b585d7a653a599a17b9"), "name" : "Romal", "age" : "20", "dept" : "I.T.",
  "pin" : "239720" }
```

3. Retrieve

```
from pymongo import MongoClient
client = MongoClient('localhost', 27017)
db = client.TYITDB239720
def read():
    try:
        Col = db.MyCol.find()
        print("\nAll data from database TYITDB239720:")
        for MyCol in Col:
            print(MyCol)
    except Exception:
        print(str(e))
```

read()

```
All data from database TYITDB239720:
{'_id': ObjectId('64d08b585d7a653a599a17b9'), 'name': 'Romal', 'age': '20', 'dept': 'I.T.', 'pin': '239720'}
```

4. delete

```
from pymongo import MongoClient
client = MongoClient('localhost', 27017)
db = client.TYITDB239720
```

```
def delete():  
    try:  
        name1 = input("Enter the name:")  
        db.MyCol.delete_one({"name":name1})  
        print("\nData deleted successfully")  
    except Exception:  
        print(str(e))
```

delete()

Enter the name:Romal

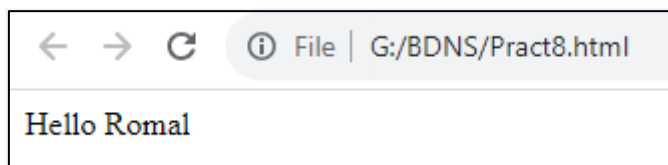
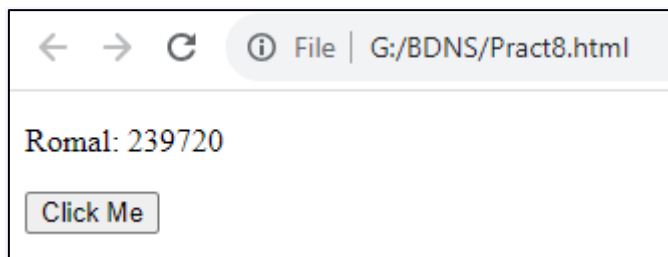
Data deleted successfully
|

```
> db.MyCol.find()  
> _
```

Practical 7

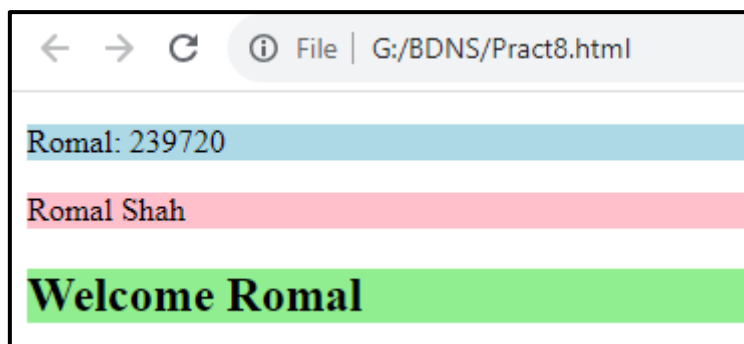
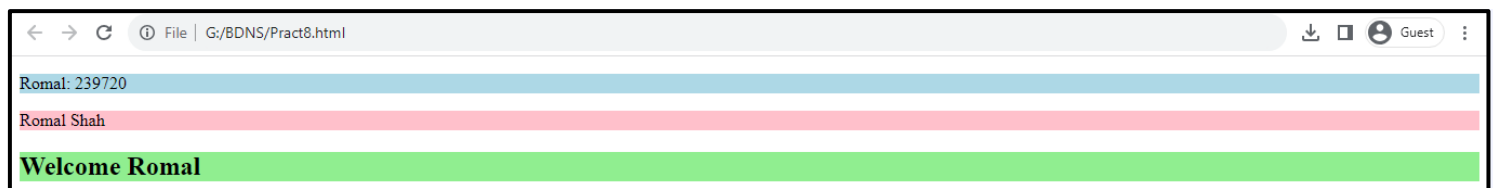
1. Write a jQuery to check the contents of the elements on the button click.

```
<!DOCTYPE html>
<head>
  <title>Basic jQuery</title>
  <script src="jquery-3.6.1.min.js"></script>
  <script>
    $(document).ready(function () {
      $("#btn").click(function(){
        document.write("Hello Romal")
      })
    })
  </script>
</head>
<body>
  <p>Romal: 239720</p>
  <button id="btn">Click Me</button>
</body>
```



2. Write a jQuery to select elements by class name, id and element names.

```
<!DOCTYPE html>
<head>
  <title>Basic jQuery</title>
  <script src="jquery-3.6.1.min.js"></script>
  <script>
    $(document).ready(function() {
      $(".class1").css("background", "lightblue");
      $("#id1").css("background", "pink");
      $("h2").css("background", "lightgreen");
    })
  </script>
</head>
<body>
  <p class="class1">Romal: 239720</p>
  <p id="id1">Romal Shah</p>
  <h2>Welcome Romal</h2>
</body>
```



3. Write jQuery to show the use of click(), hover(), on(), trigger(),off() events.

```
<!DOCTYPE html>
```

```
<head>
```

```
  <title>Basic jQuery</title>
```

```
  <script src="jquery-3.6.1.min.js"></script>
```

```
  <script>
```

```
    $(document).ready(function () {  
      $("#b1").hover (function(){  
        document.write("Hello Romal")  
      })  
      $("p").on("click", function() {  
        $(this).css("background", "yellow");  
      });  
      $("#b2").click(function(){  
        $("p").off("click");  
      });  
      $("#b3").on(function(){  
        $("#t1").hide();  
      });  
      $("input").select(function(){  
        $("input").after("Text Marked!")  
      });  
      $("#b4").click(function(){  
        $("input").trigger("select");  
      });  
    })
```

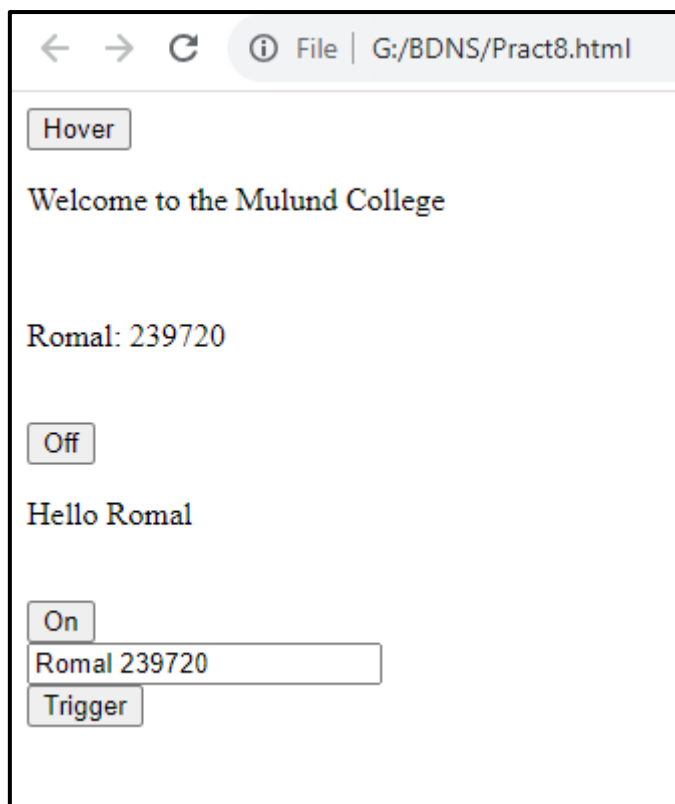
```
</script>
```

```
</head>
```

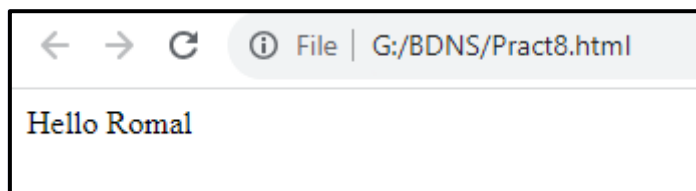
```
<body>
```

```
  <button id="b1">Hover</button><br>
```

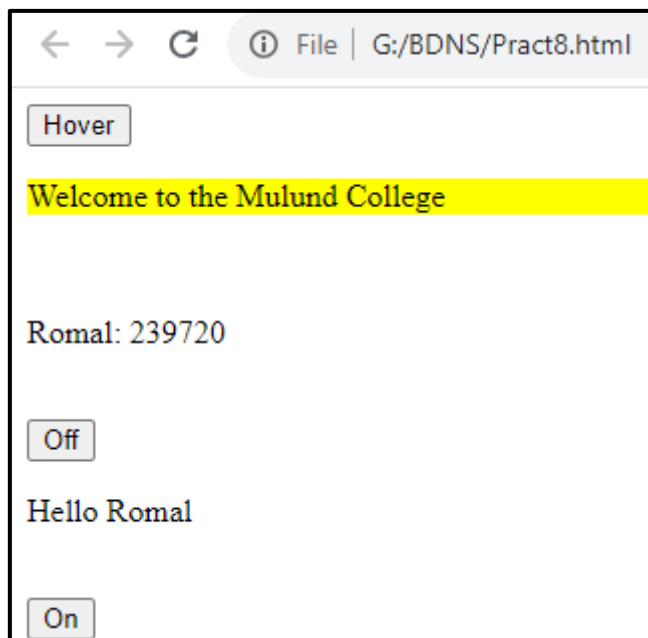
```
<p>Welcome to the Mulund College</p><br>
<p>Romal: 239720</p><br>
<button id="b2">Off</button><br>
<p id="t1">Hello Romal</p><br>
<button id="b3">On</button><br>
<input type="text" value="Romal 239720"><br>
<button id="b4">Trigger</button><br>
</body>
```



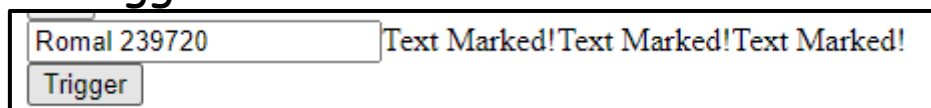
Hover:



Click, on, off



Trigger:



1. Write a jQuery to create animated show hide effect.

```
<!DOCTYPE html>
```

```
<head>
```

```
  <title>Basic jQuery</title>
```

```
  <script src="jquery-3.6.1.min.js"></script>
```

```
  <script>
```

```
    $(document).ready(function(){
```

```
      $("#b1").click(function(){
```

```
        $("p").hide();
```

```
      });
```

```
      $("#b2").click(function(){
```

```
        $("p").show();
```

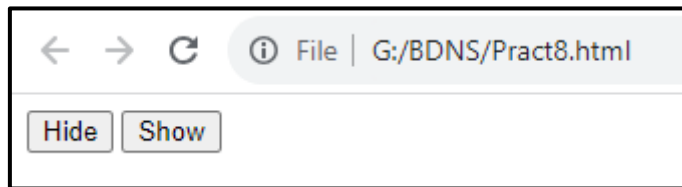
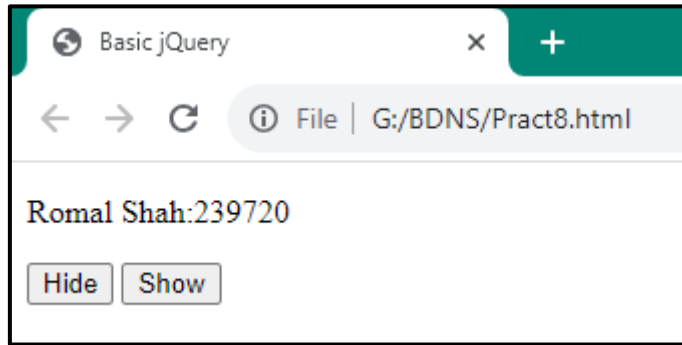
```
      })
```

```
    })
```

```
  </script>
```

```
</head>
```

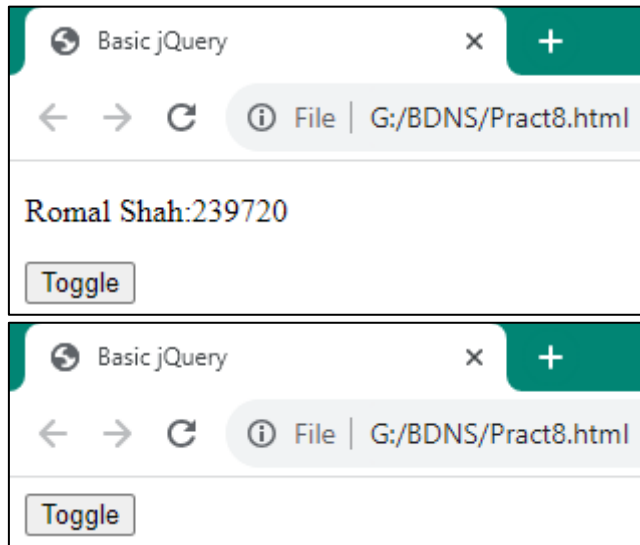
```
<body>
  <p>Romal Shah:239720</p>
  <button id="b1">Hide</button>
  <button id="b2">Show</button>
</body>
```



1. Write a jQuery to create a simple toggle effect.

```
<!DOCTYPE html>
<head>
  <title>Basic jQuery</title>
  <script src="jquery-3.6.1.min.js"></script>
  <script>
    $(document).ready(function(){
      $("button").click(function(){
        $("p").toggle();
      });
    })
  </script>
</head>
<body>
  <p>Romal Shah:239720</p>
```

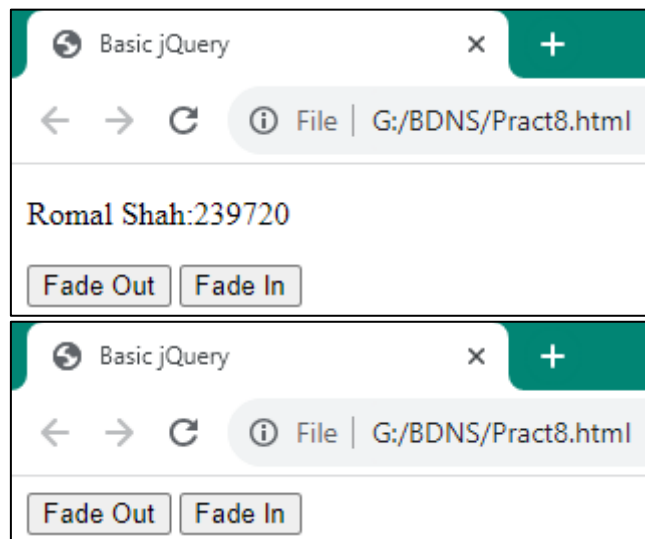
```
<button>Toggle</button>
</body>
```



1. Write a jQuery to create fade-in and fade-out effect.

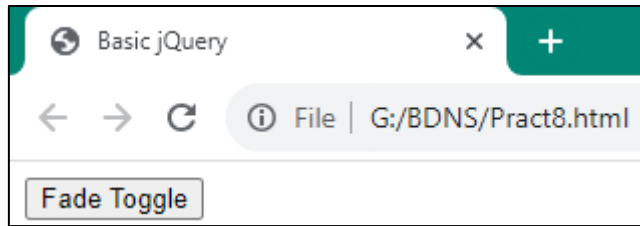
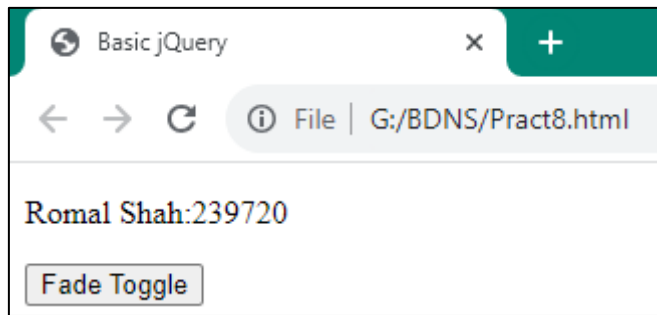
```
<!DOCTYPE html>
<head>
  <title>Basic jQuery</title>
  <script src="jquery-3.6.1.min.js"></script>
  <script>
    $(document).ready(function(){
      $(".c1").click(function(){
        $("p").fadeOut(5000);
      });
      $(".c2").click(function(){
        $("p").fadeIn(5000);
      })
    })
  </script>
</head>
<body>
  <p>Romal Shah:239720</p>
  <button class="c1">Fade Out</button>
```

```
<button class="c2">Fade In</button>
</body>
```



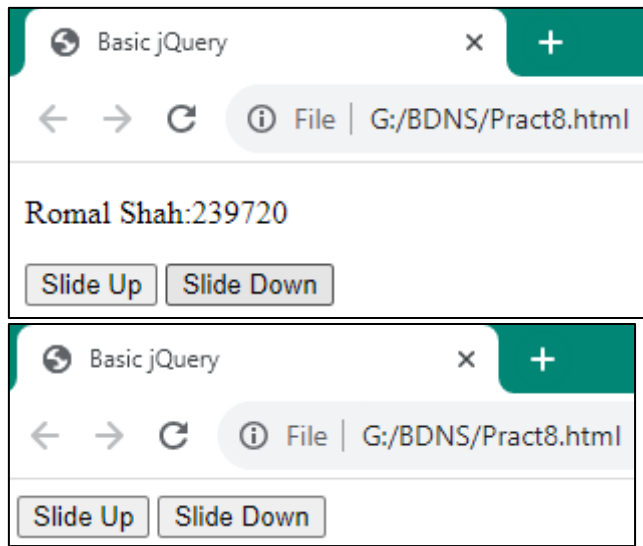
1. Write a jQuery to create fade toggle effect.

```
<!DOCTYPE html>
<head>
  <title>Basic jQuery</title>
  <script src="jquery-3.6.1.min.js"></script>
  <script>
    $(document).ready(function(){
      $("button").click(function(){
        $("p").fadeToggle(5000);
      });
    })
  </script>
</head>
<body>
  <p>Romal Shah:239720</p>
  <button>Fade Toggles</button>
</body>
```



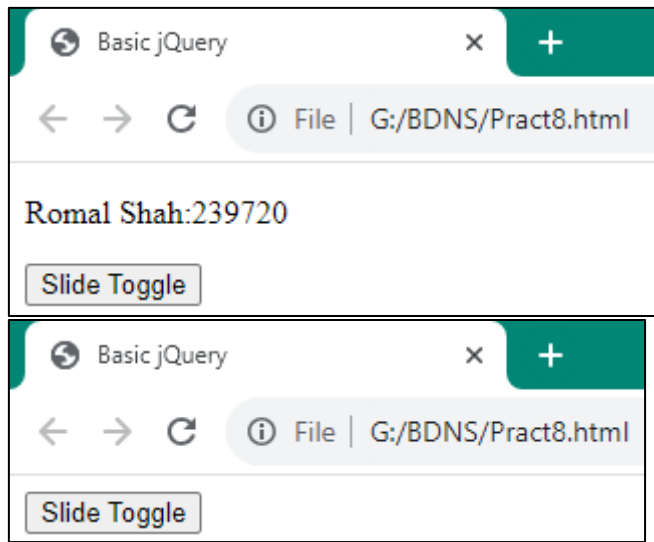
1. Write a jQuery to create slide-up and slide-down effect.

```
<!DOCTYPE html>
<head>
  <title>Basic jQuery</title>
  <script src="jquery-3.6.1.min.js"></script>
  <script>
    $(document).ready(function(){
      $("#b1").click(function(){
        $("p").slideUp(3000);
      });
      $("#b2").click(function(){
        $("p").slideDown(3000);
      })
    })
  </script>
</head>
<body>
  <p>Romal Shah:239720</p>
  <button id="b1">Slide Up</button>
  <button id="b2">Slide Down</button>
</body>
```



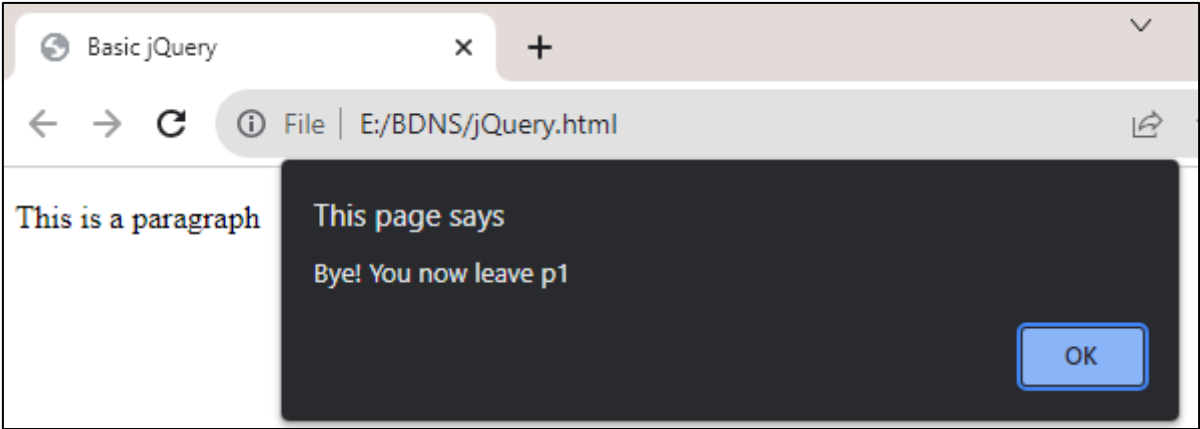
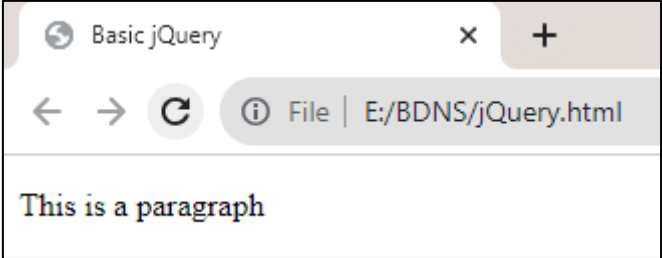
1. Write a jQuery to create slide toggle effect.

```
<!DOCTYPE html>
<head>
  <title>Basic jQuery</title>
  <script src="jquery-3.6.1.min.js"></script>
  <script>
    $(document).ready(function(){
      $("button").click(function(){
        $("p").slideToggle(5000);
      })
    })
  </script>
</head>
<body>
  <p>Romal Shah:239720</p>
  <button>Slide Toggle</button>
</body>
```



1. Write a jQuery to create Mouse Event.

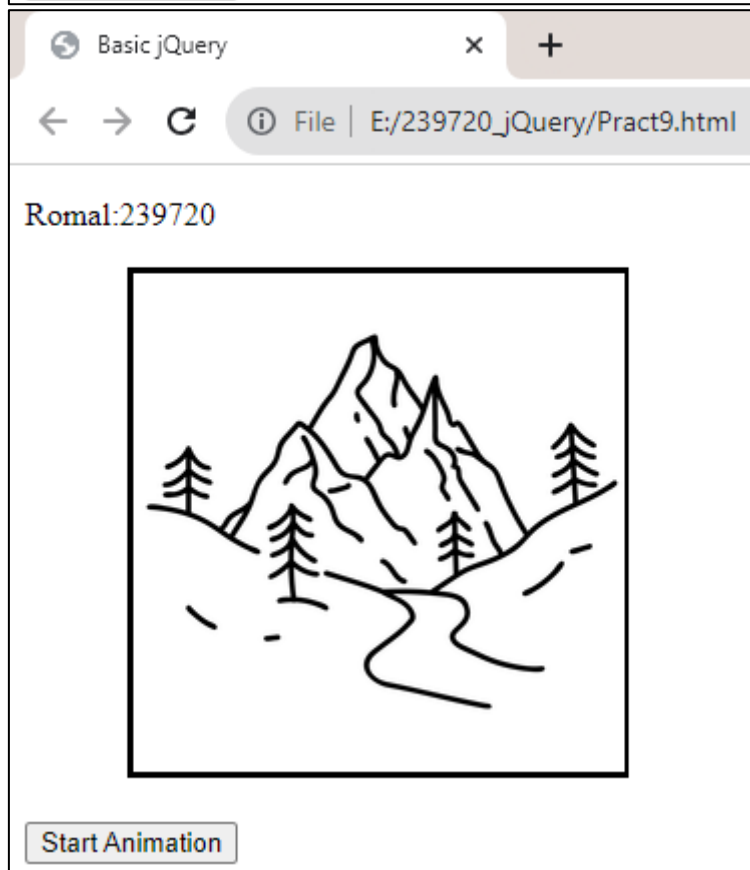
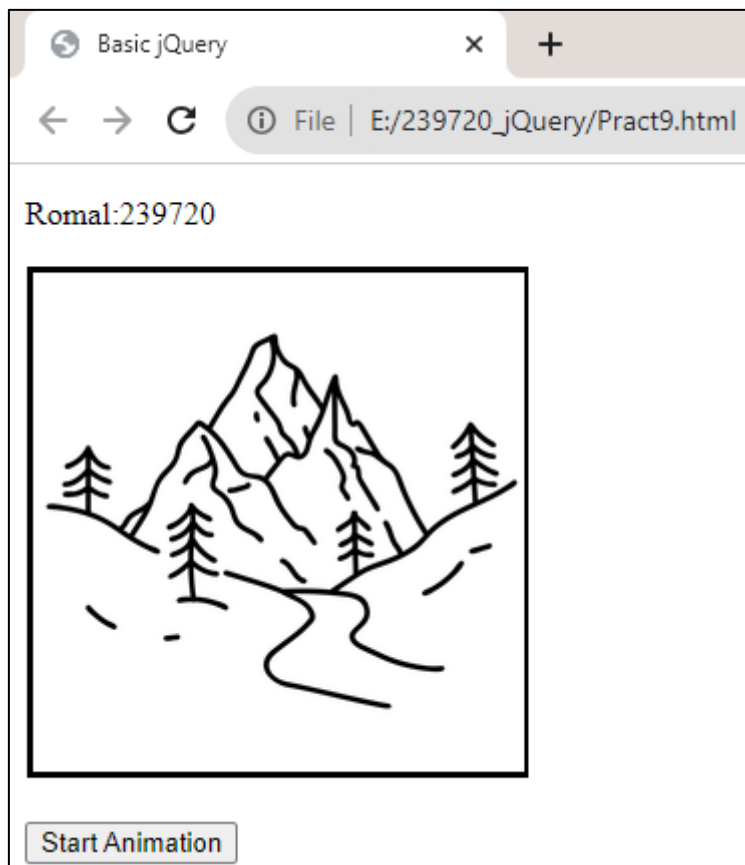
```
<!DOCTYPE html>
<html>
<head>
  <title>Basic jQuery</title>
  <script src="jquery-3.6.1.min.js"></script>
  <script>
    $(document).ready(function(){
      $("#p1").mouseleave(function(){
        alert("Bye! You now leave p1");
      })
    })
  </script>
</head>
<body>
  <p id="p1">This is a paragraph</p>
</body>
</html>
```



Practical 8

1. Write a jQuery to create animation effect.

```
<!DOCTYPE html>
<head>
  <title>Basic jQuery</title>
  <script src="jquery-3.3.1.min.js"></script>
  <script>
    $(document).ready(function(){
      $("button").click(function(){
        $("img").animate({left:50});
      })
    })
  </script>
  <style>
    img{position:relative;}
  </style>
</head>
<body>
  <p>Romal:239720</p>
  <br>
  <br><button>Start Animation</button>
</body>
```



1. Write a jQuery to animate multiple css properties.

```
<html>
```

```
<head>
```

```
<script src="jquery-3.6.1.min.js"></script>
```

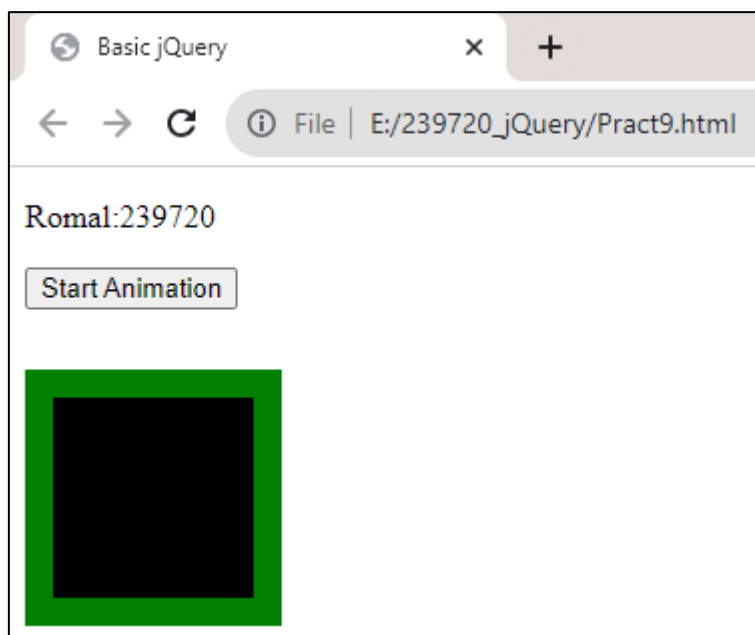
```

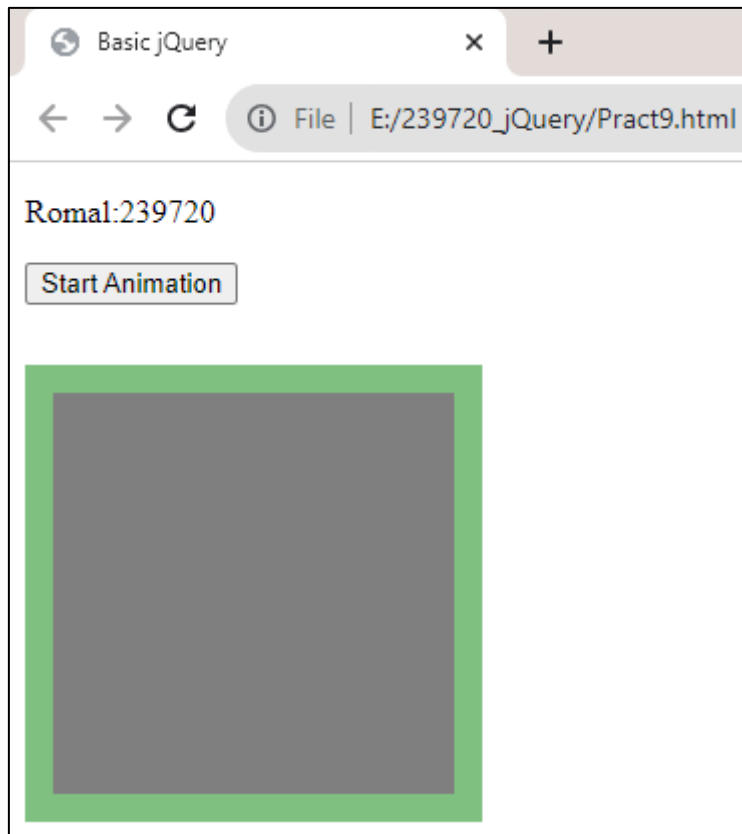
<script>
$(document).ready(function(){
    $("#b1").click(function(){

$("#div1").animate({width:"150px",height:"150px",opacity:"0.5",left:"150px"});

    })
})
</script>
<style>
#div1{width:100px;height:100px;background:yellow;position:relative;}
</style>
</head>
<body>
<p> Animation</p>
<div id="div1"></div>
<button id="b1">Click to Animate</button>
</body>
</html>

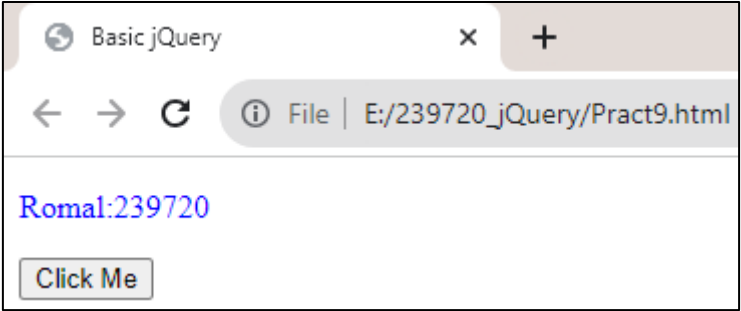
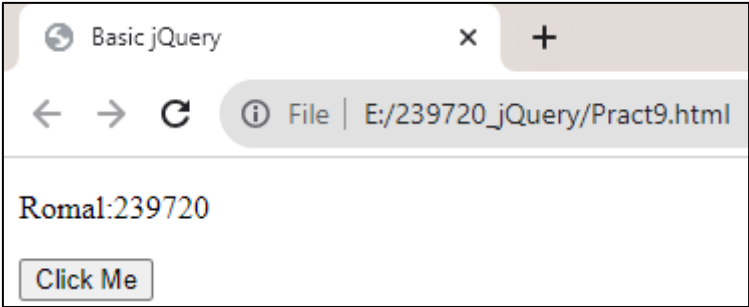
```





1. Write a jQuery to perform method of chaining.

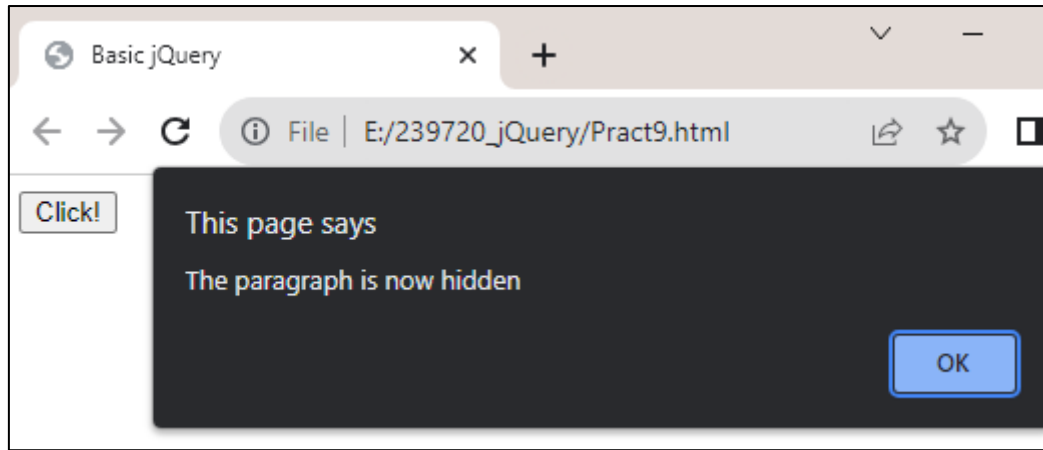
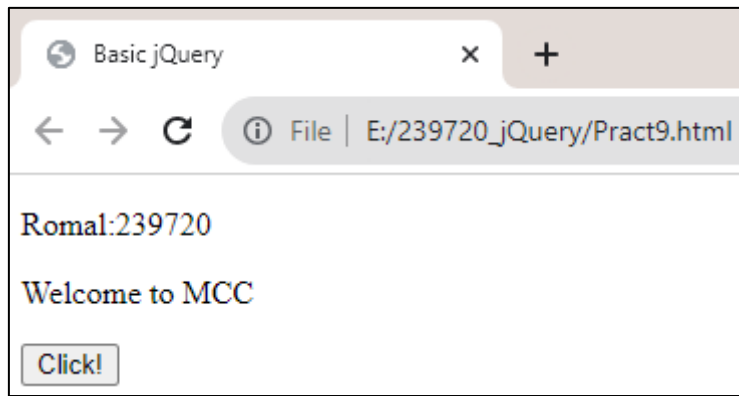
```
<!DOCTYPE html>
<head>
  <title>Basic jQuery</title>
  <script src="jquery-3.3.1.min.js"></script>
  <script>
    $(document).ready(function(){
      $("button").click(function(){
        $("#p1").css("color", "blue").slideUp(2000).slideDown(2000)
      })
    })
  </script>
</head>
<body>
  <p id="p1">Romal:239720</p>
  <button>Click Me</button>
</body>
```



Practical 9

1. Write a jQuery effect method with a callback function.

```
<!DOCTYPE html>
<head>
  <title>Basic jQuery</title>
  <script src="jquery-3.3.1.min.js"></script>
  <script>
    $(document).ready(function(){
      $("button").click(function(){
        $("p").hide("slow",function(){
          alert("The paragraph is now hidden")
        })
      })
    })
  </script>
</head>
<body>
  <p>Romal:239720</p>
  <p>Welcome to MCC</p>
  <button>Click!</button>
</body>
```



1. Write a jQuery to get and set text context of the elements.

```
<!DOCTYPE html>
```

```
<head>
```

```
<title>Basic jQuery</title>
```

```
<script src="jquery-3.6.1.min.js"></script>
```

```
<script>
```

```
$(document).ready(function(){
```

```
$("#b1").click(function(){
```

```
var str = $("p").text();
```

```
alert(str);
```

```
})
```

```
$("#b2").click(function(){
```

```
$("p").text("Hello World");
```

```
})
```

```
})
```

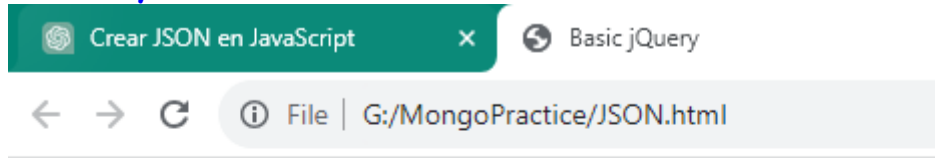
```
</script>
```

```
</head>
```

```

<body>
  <p>Romal Shah:239720</p>
  <p> MongoDB</p>
  <button id="b1">Get</button>
  <button id="b2">Set</button>
</body>

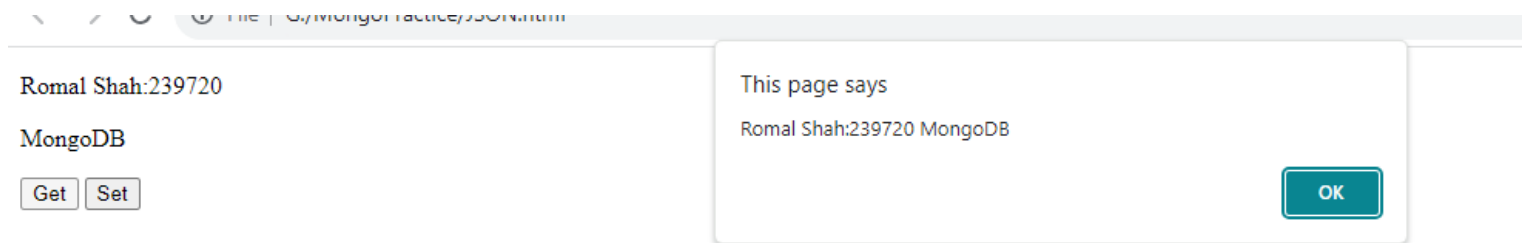
```



Romal Shah:239720

MongoDB

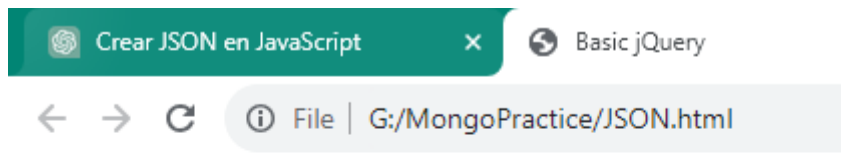
Get Set



Romal Shah:239720

MongoDB

Get Set



Hello World

Hello World

Get Set

1. Write a jQuery to get and set HTML contents of the elements.

```

<!DOCTYPE html>
<head>
  <title>Basic jQuery</title>

```

```
<script src="jquery-3.6.1.min.js"></script>
```

```
<script>
```

```
$(document).ready(function(){
```

```
$("#b1").click(function(){
```

```
var str = $("p").html();
```

```
alert(str);
```

```
})
```

```
$("#b2").click(function(){
```

```
$("p").html("<b>Hello World<b>");
```

```
})
```

```
})
```

```
</script>
```

```
</head>
```

```
<body>
```

```
<p><b>Romal Shah:</b><u>239720</u></p>
```

```
<p> MongoDB</p>
```

```
<button id="b1">Get</button>
```

```
<button id="b2">Set</button>
```

```
</body>
```

← → ↻ ⓘ File | G:/MongoPractice/JSON.html

Romal Shah:&u239720

MongoDB

Get

Set

Romal Shah:&u239720

MongoDB

Get

Set

This page says

Romal Shah: <u>239720</u>

OK

Hello World

Hello World

Get

Set

10. JSON

Creating JSON

```
<html>
<head>
<title>Creating JSON</title>
</head>
<body>
<script type="text/javascript">
    var data={
        "name":"Roma1",
        "age":20,
        "dept":"IT"
    }
    document.writeln(JSON.stringify(data)+"<br>");
    document.writeln(JSON.stringify(data,["name","age","dept"])
+"<br>");
    document.writeln(JSON.stringify(data,["name","age","dept"],
5));
</script>
</bo
dy>
</html>
```

Output:

← → ↻ ⓘ File | G:/MongoPractice/JSON.html

```
{"name":"Roma1","age":20,"dept":"IT"}
{"name":"Roma1","age":20,"dept":"IT"}
{ "name": "Roma1", "age": 20, "dept": "IT" }
```

Parsing JSON

<html>

<body>

<script type="text/javascript">

var data = '{"name":"Romal","age":20,"dept":"IT"}';

var jsonObj = JSON.parse(data, function(name, value) {
 return value;

});

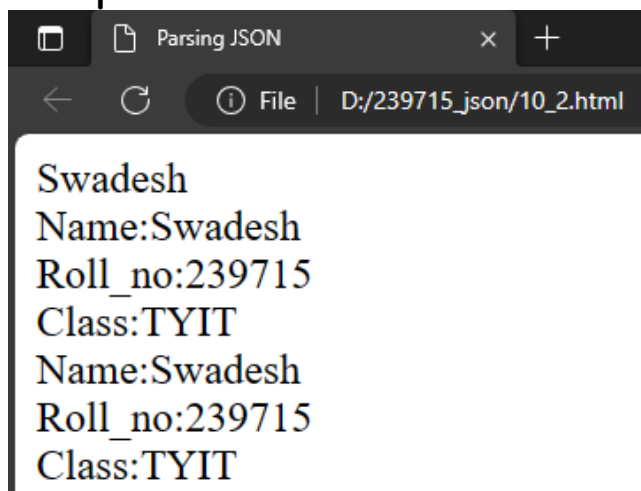
```

document.writeln(jsonObj.name);
for (key in jsonObj) {
    document.writeln("<br/>" + key + " : " + jsonObj[key]);
}

var jsonObj = JSON.parse(data, function(name, value) {
    if (name == "sem") {
        return undefined;
    } else {
        return value;
    }
});
for (key in jsonObj) {
    document.writeln("<br/>" + key + " : " + jsonObj[key]);
}
</script>
</body>
</html>

```

Output:



3.

<html>

<body>

<script type="text/javascript">

var data = '{"name":"Romal","age":20,"dept":"IT"}';

```
var obj = JSON.parse(data);  
for(var key in obj)  
{  
document.writeln("<br/>"+"key+"+"":"+obj[key]);  
}
```

</script>

</body>

</html>

Output:

name: Romal
age: 20
dept: IT